

Your voice, working with you.

An interview-ready explanation of Parvathi, from first principles to the important engineering tradeoffs.

THE 30-SECOND PITCH

Parvathi is a native voice assistant for macOS and Windows. It lets you dictate into another app, issue a small set of computer commands, and ask questions with spoken replies. It separates dictation from execution, validates actions before dispatch, and checks results wherever the operating system exposes reliable evidence.

What problem does it solve?

Switching between typing, launching applications, searching, and changing small settings interrupts a working session. Parvathi provides explicit voice modes and a compact feedback panel so those tasks can start with a shortcut and finish without navigating a large interface.

What did you actually build?

The Mac application uses SwiftUI/AppKit, Apple Speech, Accessibility, CoreAudio, Keychain, and native speech. A separate Windows C#/WPF application uses offline Vosk recognition, Windows accessibility and audio APIs, DPAPI, and SAPI speech. Both use explicit modes and optional OpenAI or Ollama text generation. Windows has self-contained installer and portable packaging.

What is the most important design decision?

The user chooses the mode before recording. In Dictation, "open Chrome" remains text. Only Command mode reaches the typed action registry. AI output is used for conversation or editing; it never becomes a shell command or an automatically executed action.

How should I present the project honestly?

Describe the native Mac MVP and Windows prerelease separately. Build and unit-test evidence are different from live microphone and cross-application evidence. Windows interactive checks remain unperformed. Do not claim broad editor compatibility, perfect speech accuracy, live web answers, a signed Windows release, or production distribution readiness.

Reading map: pages 2-11 explain the original macOS implementation. Pages 12-15 explain the Windows architecture, privacy, installation, GitHub release workflow, and interview demonstration.

Explain it from scratch.

The beginner-friendly model: ears, a switchboard, hands, and a voice.

What is speech recognition?

It turns an incoming microphone signal into words. AVAudioEngine produces small audio buffers. SFSpeechAudioBufferRecognitionRequest receives them, and Apple Speech returns partial text followed by a final transcription. The app does not save an audio recording to disk.

What does the language model do?

It answers questions, polishes dictation, rewrites a selection, or explains selected text. It is not necessary for verbatim dictation or the supported commands. An AIProvider protocol keeps the caller independent of the OpenAI and Ollama HTTP formats.

How does the assistant control the Mac?

It calls specific native APIs. NSWorkspace opens applications and websites. CoreAudio adjusts volume and mute state. Accessibility identifies text fields and inserts text; a guarded clipboard fallback can send paste to the captured process. No generated shell script sits between the transcript and the operating system.

Why SwiftUI and AppKit instead of a web shell?

The first development machine was a Mac, and the most sensitive integrations are native: menu-bar behavior, permissions, shortcuts, Accessibility, Keychain, and audio devices. SwiftUI handles the interface and AppKit handles the floating panel. These frameworks do not run on Windows, so that platform has a separate C#/WPF implementation described on pages 12-15.

What is the difference between a mode and an intent?

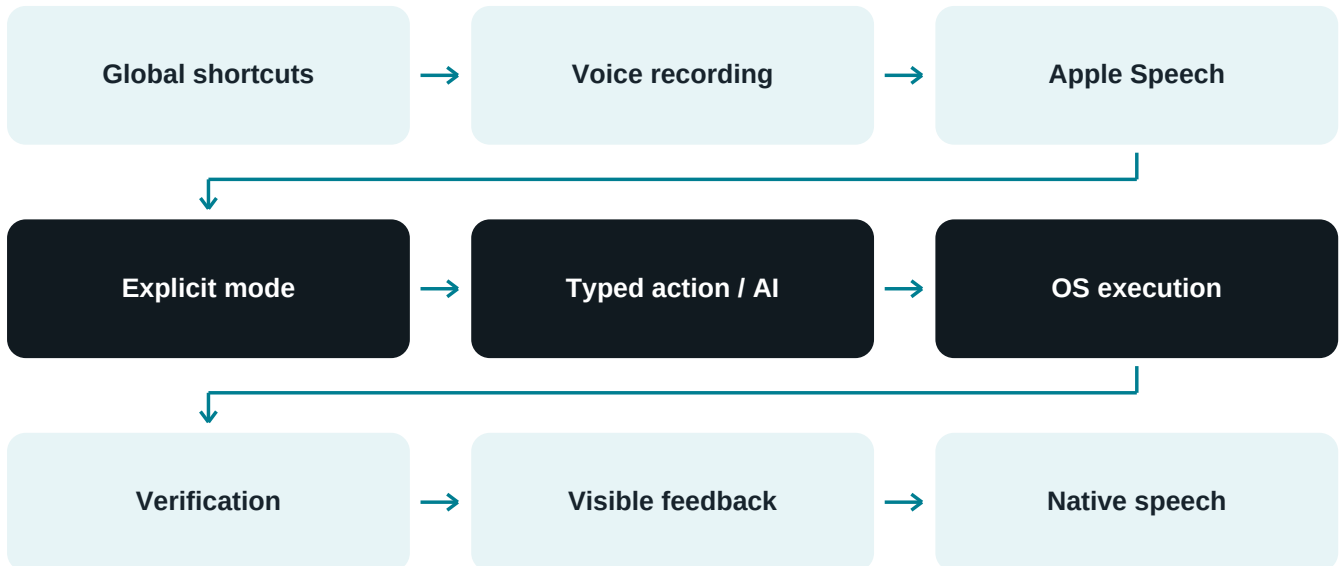
Mode is the user-selected interaction contract: dictate, command, ask, rewrite, or explain. Intent is a validated action within Command mode, such as setVolume(30). Keeping these separate prevents a phrase inside a paragraph from unexpectedly launching an application.

A CONCRETE EXAMPLE

Hold Control-Option-Space in TextEdit and say "open Chrome" to insert those words. Press Control-Option-C to start a command recording, say the same words, then press it again to request the installed Chrome application.

Architecture with clear boundaries.

The controller owns request state; each service owns one operating-system or provider boundary.



The diagram shows the logical pipeline. Dictation bypasses AI in verbatim mode; conversation bypasses OS execution. Each route rejoins visible feedback, and only appropriate replies are spoken.

Where does the state machine live?

AssistantController coordinates idle, listening, transcribing, thinking, executing, and speaking. RequestGate identifies the current request and rejects stale completions. The floating panel and dashboard observe this state instead of independently launching work.

Why separate ParvathiCore from the app?

The core contains modes, typed command actions, routing, application-name resolution, and request ownership. Those rules can be tested without a microphone, Accessibility permission, credentials, or a GUI. The app target contains the native services and SwiftUI views.

Are the modules really replaceable?

AIProvider, SpeechRecognizing, SpeechSynthesizing, and SystemAutomating define replaceable contracts. Recognition and speech output can be injected into the controller; native implementations are the defaults. Focus insertion remains an isolated platform service. Expanding injected fixtures is a useful next step for integration testing.

Walk through one complete request.

Trace a paragraph from the shortcut to a verified edit in another application.

- 1. Capture intent and destination.** The shortcut selects Dictation. Before showing the panel or requesting microphone access, FocusService snapshots the foreground process, focused element, selected range, current text, and activation revision.
- 2. Start a request.** RequestGate assigns a UUID. The controller cancels older work, installs partial-text and level callbacks for that UUID, and shows the recording state. Permissions are requested before audio starts.
- 3. Stream audio.** The microphone feeds an AVAudioEngine tap. Audio buffers go directly to Apple Speech. A root-mean-square signal level drives the waveform. The recording is capped at 55 seconds.
- 4. Finalize words.** Releasing the push-to-talk shortcut ends audio. Hands-free recording ends on the next shortcut press. The service waits up to four seconds for a final result. An incomplete result is not silently inserted.
- 5. Route without guessing.** ModeRouter returns dictation text. Verbatim uses the recognized words; Polished makes a separate AI editing request. A cancellation and request-ownership check follows each suspension point.
- 6. Validate and execute once.** FocusService rechecks the original destination, text, and cursor. RequestGate claims the action before dispatch. Accessibility insertion is preferred, with guarded paste only when the editor does not support direct insertion.
- 7. Verify and report.** The expected resulting field value is compared with what Accessibility reads back for up to one second. The UI reports verified insertion or a sent-but-unverified result. There is no automatic retry after dispatch.

Where can this flow stop?

Missing permissions, an unsupported locale, no editable target, a focus change, canceled work, network failure, missing AI credentials in Polished mode, or missing final transcription all end with an actionable message. A side effect already dispatched to macOS cannot be rolled back by Escape.

Code tour and command algorithms.

Use these file names when an interviewer asks where a behavior is implemented.

App/ParvathiApp.swift Creates the dashboard, settings scene, menu-bar controls, and shared controller.

Stores/AssistantController.swift Coordinates record, transcribe, route, execute, verify, and reply; keeps bounded in-memory conversation.

ParvathiCore/ModeRouter.swift Enforces the mode boundary. Dictation never calls CommandRouter.

ParvathiCore/CommandRouter.swift Matches a small grammar, validates arguments, and returns CommandAction values.

ParvathiCore/ApplicationResolver.swift Resolves exact names and bundle IDs, known aliases, prefixes, then substrings; ambiguous matches request a full name.

Services/FocusService.swift Captures text destinations, validates focus, preserves clipboard content, and checks insertion results.

Services/SystemAutomation.swift Discovers installed applications and dispatches only supported native actions.

How does the parser avoid dangerous interpretation?

It recognizes a deliberately small command language rather than arbitrary generated code. Text after "Type:" is data. Volume is an integer from 0 to 100. Website URLs allow HTTP or HTTPS and reject embedded credentials. Unsupported actions and multi-step commands produce guidance instead of improvisation.

What happens for "Open Chrome"?

The parser returns openApplication("Chrome"). Discovery builds a catalog from standard application folders and running applications. The resolver knows the Chrome alias, but still requires an actual installed match. NSWorkspace opens or focuses that bundle; the service checks that the expected process becomes foreground.

INTERVIEW NUANCE

A deterministic grammar reduces the MVP attack surface and makes tests predictable. Its cost is limited phrasing. Future natural-language planning should produce the same typed actions and still pass the same validation boundary.

Focus, clipboard, and cancellation.

The difficult desktop problems live between applications and asynchronous callbacks.

Why is remembering only the application name unsafe?

The user can move to another field in the same app, change the selection, or switch away and back while transcription is pending. Parvathi compares the process, Accessibility element, selected range, original value, and application-activation revision before dispatch. A mismatch blocks insertion.

How do you preserve the clipboard?

The fallback saves every available data representation for every clipboard item, checks that the clipboard did not change during capture, writes the insertion text, and posts Command-V to the captured process. A deferred restore runs only if the clipboard still has the change count owned by Parvathi. A newer user copy is never overwritten. Lazy representations that cannot be read cause a safe failure.

How does Escape work across the pipeline?

The controller invalidates RequestGate, cancels Swift tasks and timers, ends microphone capture, cancels recognition, and stops native speech immediately. Request IDs prevent late callbacks from changing a newer session. URLSession requests inherit task cancellation. The stop button provides the same application-level action.

What does duplicate protection guarantee?

An action can be claimed only once for the current request. Repeated completion handlers and stale tasks cannot dispatch it twice. The app deliberately avoids automatic retries after an uncertain result. This is an in-memory per-request guarantee, not durable exactly-once delivery across crashes; a fresh user request is a new action.

Can cancellation undo an action?

No. It prevents pending work and further activation where possible. A paste event, a browser-open request, a launched process, or a volume change already handed to macOS may still take effect. Feedback states this explicitly so the user checks the result before repeating an action.

KNOWN COMPATIBILITY LIMIT

Strict validation is intentionally conservative. Some browser editors, rich document apps, remote desktops, and custom controls do not expose the required text or cursor attributes. Parvathi leaves the transcript visible for manual copying instead of guessing the target.

Providers, privacy, and trust.

Understand exactly which information can leave the device.

What are the defaults?

On-device-only Apple Speech, verbatim dictation, and transcript history disabled. If the selected language or device does not support on-device speech, the app reports that limitation. It does not silently fall back to server recognition. The microphone follows macOS Sound settings.

When does audio leave the Mac?

If the user explicitly turns off On-device only, Apple Speech may use Apple servers. Parvathi itself never writes raw audio files. The OpenAI and Ollama text providers receive text, not microphone audio. Native speech output uses the system speech synthesizer.

What does the AI provider receive?

Ask mode sends the prompt and bounded recent conversation. Polished dictation sends the transcript and editing instruction. Rewrite and Explain send the explicitly selected text and user instruction. OpenAI uses POST /v1/responses with store:false. Ollama uses POST /api/chat with stream:false. A configured remote endpoint receives the supplied text; a loopback endpoint stays on this machine at the HTTP boundary.

Does store:false mean zero retention?

No. It disables response storage for later retrieval through the API; it is not a blanket statement about provider retention or abuse-monitoring policies. Review the selected provider policies and account controls before using sensitive information. The client does not make stronger promises than its controls support.

Where are keys and transcripts kept?

Credentials use macOS Keychain. Ordinary preferences use UserDefaults. Optional history stores the latest 100 completed requests in Application Support/Parvathi/history.json with restricted file permissions. History is not encrypted by this app. Turning it off clears the file; Clear session removes local conversation and saved history.

How do you handle prompt injection?

Selected text and external excerpts are described as untrusted data in provider instructions. Crucially, model output has no route to the OS action executor. Editing output can still be incorrect, so users should review it. There is no screen capture, arbitrary shell tool, or retrieval tool in this MVP.

Verification and useful failures.

Success is a claim that needs evidence, not a cheerful sentence.

Which outcomes can be verified?

Application activation checks the foreground process. Text insertion compares the editor value with the expected result. Volume and mute read device properties back. Browser opening only confirms that macOS accepted the URL request, so it stays unverified with respect to actual page loading. Unsupported hardware controls return an error.

How are timeouts bounded?

Recording stops at 55 seconds; final transcription waits four seconds. AI uses a 30-second request timeout and 45-second resource timeout. App launch waits up to ten seconds, activation verification two seconds, and insertion verification one second. Spoken output has a 60-second watchdog. Accessibility messages use short IPC timeouts. These limits bound waiting, not already-dispatched macOS effects.

What does a helpful error look like?

A denied microphone permission names System Settings > Privacy & Security > Microphone. Missing AI credentials point to Settings rather than blocking verbatim mode. A changed cursor says that nothing was inserted. An uncertain paste says to check the field before retrying. HTTP errors distinguish rejected credentials, a missing model, rate limits, and connectivity.

How do provider requests protect data in transit?

Base URLs require HTTPS except exact loopback HTTP addresses. URLs with embedded credentials, queries, or fragments are rejected. Requests use an ephemeral URLSession without cookies or a persistent cache. Redirects are refused. Server error bodies are not echoed because they could contain secrets or user text.

What do automated tests establish?

The development run passed 33 tests across five suites. Core tests cover modes, command validation, ambiguity, cancellation, and duplicate action claims. Service tests use controlled HTTP responses to verify parsing, errors, cancellation, and credential isolation. These tests do not prove recognition quality, every editor integration, or a live provider account.

EVIDENCE STANDARD

The packaged app launched with native window metadata, and its dashboard render was inspected. Real Chrome activation was requested but correctly reported unverified while macOS loginwindow remained foreground. Retest in an unlocked session. Live microphone insertion and authenticated-provider tests remain unrun.

Build and run it from scratch.

A developer setup, followed by a dependable first demonstration.

What do I need?

A Mac running macOS 14 or later, current Apple Command Line Tools with Swift 6.0 or later, and a microphone. The app has no third-party Swift packages. This implementation was developed on an Apple Silicon Mac with Swift 6.4. AI is optional for the core dictation and command paths.

```
xcode-select --install
cd Parvathi
swift build
./scripts/test.sh
./script/build_and_run.sh
```

Why use the application bundle to run it?

The packaging script creates dist/Parvathi.app with microphone and speech usage descriptions. macOS permission handling needs a stable app identity. Launch the bundle instead of relying on swift run for an interactive permission test. scripts/test.sh supplies Swift Testing paths for standalone Command Line Tools; plain swift test may fail when XCTest is absent. Grant Accessibility, Microphone, and Speech Recognition as requested, then focus a TextEdit text field.

How do I configure AI?

Open Settings and choose a provider. For OpenAI, enter <https://api.openai.com/v1>, an available model, and your key through the Keychain control. For Ollama, start its server, install a model, and choose <http://localhost:11434> plus that model name. Local Ollama needs no key; the OpenAI key is never forwarded to it. Authenticated remote Ollama is unsupported. The .env.example file is documentation only; the app does not read .env automatically.

How do I package the project?

Run ./scripts/package.sh for a release application and dist/Parvathi-macOS.zip. The script uses ad-hoc signing unless SIGNING_IDENTITY is supplied. Distribution outside the developer machine still needs an appropriate Developer ID, hardened-runtime review, notarization, and testing on the intended Mac architectures.

SIMPLE DEMO SEQUENCE

1. Dictate a short paragraph into TextEdit.
2. Dictate "open Chrome" and show that it stays text.
3. Use Command mode to open Chrome.
4. Show a changed-focus refusal.
5. Ask a short question with a configured provider, then interrupt speech with Escape.

Test the actual user experience.

Reproducible acceptance scenarios for a reviewer or interviewer.

Microphone and insertion. Use a blank TextEdit document. Hold Control-Option-Space, speak a paragraph, release, and compare the exact destination text. Repeat with hands-free mode. Watch for a visible microphone-active state and a final verified or unverified outcome.

Mode isolation. Speak "open Chrome" in Dictation and confirm no application launch. Repeat through Control-Option-C in Command mode and observe Chrome opening or focusing. If Chrome is absent, a real not-found error is the expected result.

Focus safety. Begin dictation in one field, change the cursor, switch applications, or edit the text before finishing. The request should refuse insertion. Confirm that neither the original nor the new target was unexpectedly modified.

Clipboard and selected text. Put formatted text or an image on the clipboard, exercise a compatible paste fallback, and verify preservation. Select a paragraph and choose Rewrite selection; request a shorter version. Use Explain selection and confirm it answers without replacing the selection.

Stop and error paths. Press Escape during recording, AI waiting, and speech. Disable permission and check the recovery message. Use an invalid key, unreachable provider, ambiguous application name, and unsupported volume device where available. Do not mark an unavailable scenario as passed.

Privacy and UI. Leave history off and confirm no completed request is retained after restart. Enable history and then delete it. Test keyboard navigation, VoiceOver labels, and Reduce Motion. Confirm only the permitted mode sends relevant text to the provider.

What should I record as evidence?

Date, OS and architecture, app build or commit, input and output devices, editor, provider/model where relevant, exact steps, expected result, observed result, and Pass/Fail/Not run. Use screenshots only with explicit consent, and avoid capturing keys or personal document content.

What limitations should I mention before a demo?

English command grammar, Apple Speech language availability, strict Accessibility compatibility, no wake word or screen understanding, no multi-step actions, no live answer retrieval, and no notarized public installer. Voice and provider performance depend on the machine, permissions, model, and network.

Questions you should be ready for.

Short answers with concrete tradeoffs, followed by the official API references.

What was the hardest engineering problem?

Maintaining a trustworthy destination while recognition and AI complete asynchronously. The solution is an immutable focus snapshot, repeated validation, per-request ownership, and honest unverified feedback when the OS cannot prove the final result.

How would you add more natural commands?

Add a planner that returns a constrained typed action, then reuse the same validators and executor. Keep Dictation isolated. Add clarification for uncertain targets. Introduce bounded multi-step plans only after individual actions have reliable verification and cancellation semantics.

How would you add sensitive capabilities?

Sending messages, deleting files, purchasing, or changing sensitive settings would need a concrete preview and explicit confirmation before dispatch, plus scoped permissions and audit evidence. None of those actions is currently implemented.

What would you improve next?

Expand injected service fixtures, broader editor tests, configurable input-device routing, richer clarification, signed distribution, and performance measurements. Wake-word activation and visible, explicit screen capture are later work, not current product claims.

How would you measure success?

Measure time to first partial transcript, end-of-speech to insertion latency, cancellation response time, verified-action rate, focus-refusal rate, and manual word-error rate on consented test phrases. This release does not invent benchmark numbers.

OFFICIAL REFERENCES

[Apple Speech audio buffers](https://developer.apple.com/documentation/speech/sfspeechaudiobufferrecognitionrequest) - <https://developer.apple.com/documentation/speech/sfspeechaudiobufferrecognitionrequest>

[Apple Accessibility](https://developer.apple.com/documentation/applicationservices/axuielement) - <https://developer.apple.com/documentation/applicationservices/axuielement>

[Apple application launching](https://developer.apple.com/documentation/appkit/nsworkspace) - <https://developer.apple.com/documentation/appkit/nsworkspace>

[OpenAI text generation](https://developers.openai.com/api/docs/guides/text) - <https://developers.openai.com/api/docs/guides/text>

[OpenAI response storage behavior](https://developers.openai.com/api/docs/guides/migrate-to-responses) - <https://developers.openai.com/api/docs/guides/migrate-to-responses>

[Ollama chat endpoint](https://docs.ollama.com/api/chat) - <https://docs.ollama.com/api/chat>

Why a separate Windows app?

Keep the same product contract while using the operating system APIs that actually exist.

Why C#, .NET, and WPF?

Windows requires native tray, keyboard, audio, accessibility, and secure-storage integration. WPF provides a native desktop UI while C# calls Windows APIs directly. The project pins .NET SDK 10.0.401 and runtime 10.0.12. The Mac source remains in its existing Swift targets; the Windows source lives under windows/.

Which layers are shared conceptually?

Both implementations separate recognition, reasoning, speech output, focus validation, and operating-system actions. Windows uses IRecognitionService, IAIProvider, ISpeechOutput, IFocusService, and ISystemAutomation. Typed actions and explicit modes preserve the safety contract without attempting to compile Apple frameworks for Windows.

How does offline dictation work?

NAudio records 16 kHz, 16-bit mono PCM in short-lived buffers. Vosk turns the buffers into partial words, finalized segments, and a final result. No raw audio file is saved. Users explicitly download the English recognition model once. Basic transcription then needs no paid API key or separately running model server.

What protects the model download?

The official archive is 41,205,931 bytes. The installer checks its exact size and pinned SHA-256, rejects unsafe archive paths and links, bounds extraction, and installs from a staging directory. A per-file manifest supports integrity checks before recognition. Network failures and cancellation remove scratch downloads.

How do you insert into the correct Windows field?

Capture foreground process and window, focused UI Automation element, document text, selection, and focus revision. Revalidate everything immediately before dispatch. Address a compatible native Edit/RichEdit control or ValuePattern directly. If the field cannot be verified or addressed safely, refuse insertion and retain the visible transcript.

A DELIBERATE WINDOWS TRADEOFF

The Windows adapter does not touch the clipboard or send a blind paste shortcut. This preserves every clipboard format but limits editor compatibility. Notepad and compatible accessible fields are the first acceptance targets; actual Notepad voice interaction still needs an interactive Windows test.

Windows privacy and interruption.

Explain the microphone, credentials, provider boundary, and cancellation without overclaiming.

Where does my information go?

Microphone audio stays in memory and the local Vosk decoder. The first model download contacts alphacephei.com; it does not upload speech. Optional AI modes send text and bounded context to the configured provider. A remote endpoint receives that data remotely; local Ollama requires its own running model server. There is no screenshot or retrieval tool.

How are Windows credentials stored?

The OpenAI-provider key is encrypted with DPAPI scoped to the current Windows user and saved in the local profile. It is separate from nonsecret settings and never sent to Ollama. A configurable OpenAI-compatible endpoint receives that credential, so the user must intentionally trust the selected endpoint. No key belongs in source, logs, or installer assets.

Where are settings and history?

Under %LOCALAPPDATA%\Parvathi. Settings are JSON; the credential is encrypted; the recognition model has its own directory. History is off by default and limited to 100 bounded entries when enabled. History itself is not separately encrypted. Disabling history or selecting Delete history clears it; removing the credential is a separate setting.

Does self-contained mean completely offline?

No. It means the Windows executable ships with its .NET runtime and required native libraries. Local recognition works offline after model installation, and Windows SAPI speaks locally. Optional OpenAI requires network access and an account. Ollama needs a separately installed server and model. These are distinct deployment and privacy questions.

How does Escape stop the pipeline?

The coordinator invalidates the request and propagates cancellation. Microphone stop uses a separate lock, avoiding a wait behind native decoding. A canceled native model load is fenced from starting capture afterward. Provider calls are canceled, late results are rejected, and SAPI cancels the specific active prompt. Native effects already dispatched cannot be rolled back.

Which controls prevent prompt injection?

Dictation never routes to commands. Commands use a small validated registry. AI responses, selected text, and document excerpts are data and cannot invoke the executor. Password fields, protected targets, and elevated applications are refused. There are no arbitrary scripts, file deletion, messages, purchases, or sensitive-setting tools.

Download, build, and release.

A real Windows application must survive the trip from source code to another person's computer.

How does an end user install it?

Open the repository Releases page using an account with access, download Parvathi-Setup-0.2.0-win-x64.exe, run the per-user installer, and open Parvathi from Start. Alternatively, extract the complete portable ZIP and run Parvathi.exe. No SDK, Visual Studio, Python, Node.js, Git, or terminal is required for normal use.

What is required on first launch?

Download the offline English model in Settings and select a microphone. Allow desktop microphone access in Windows privacy settings. Focus an editable field, hold Ctrl+Alt+Space, speak, and release. Ctrl+Alt+C controls command recording; Ctrl+Alt+A controls Ask. Escape and Stop interrupt active requests. Closing the dashboard leaves the tray app running.

What makes the package self-contained?

The publish folder includes the .NET desktop runtime, application assemblies, icons, and all Vosk native DLLs. Inno Setup creates the per-user installer and normal uninstall entry; a ZIP contains the same runnable folder. Keep the complete folder together. Startup with Windows is optional. Upgrades and uninstall preserve the user profile as documented.

How can a developer rebuild it?

Use Windows and the SDK pinned in windows/global.json. The scripts restore locked dependencies, build, run tests, publish win-x64, install the pinned installer compiler, and create the package. macOS can compile many checks, but Windows GitHub Actions builds the release and performs available Windows checks.

```
pwsh ./windows/scripts/build.ps1
pwsh ./windows/scripts/install-inno.ps1
pwsh ./windows/scripts/package.ps1 -Version 0.2.0
```

How is the release trustworthy?

The release workflow builds the exact tag, requires successful checks, generates SHA-256 checksums, and uploads real installer and ZIP assets to GitHub Releases. The repository stays private. Optional signing uses GitHub secrets. Without a certificate, this is explicitly an unsigned prerelease; it must not be described as trusted by Windows or as having SmartScreen reputation.

[Windows release: v0.2.0-windows.1](#) | Windows 11 x64 | Read the release notes and checksums before installing.

Defend the Windows implementation.

Interview questions, a short demonstration, and the limits of the current evidence.

What has been tested so far?

Development passed 95 C# core tests and cross-built the complete WPF application with zero warnings and errors. The Windows CI record adds adapter and packaging checks for the release commit. These results do not prove microphone quality or cross-application behavior. No interactive Windows desktop was available; live voice acceptance remains Not run.

What should the Windows demonstration show?

Install on a clean Windows 11 x64 machine, dictate a paragraph into Notepad, then dictate "open Chrome" without launching it. Say the same phrase in Command mode to open or focus installed Chrome. Change focus during transcription and show refusal. Interrupt recording, a provider request, and speech with Escape. Verify clipboard content remains unchanged.

What failures would you deliberately demonstrate?

Missing model, disconnected or denied microphone, an ambiguous app name, unavailable Chrome, an unsupported text control, an elevated target, a missing API key, and a network failure. The app must give useful recovery text and must never label a simulated or uncertain action as completed. Test upgrades and normal uninstall separately.

Why not make every editor work with a paste shortcut?

A universal-looking fallback can type into the wrong place when focus changes or a control hides its selection. This implementation uses addressable text controls and verification. Compatibility is narrower, but the refusal is honest. Expand coverage with evidence from specific editor versions before loosening the boundary.

What would you improve after the prerelease?

First collect live Windows acceptance evidence and recognition latency measurements. Then test more editors, improve multilingual recognition, and provide signed distribution. Native ARM64 needs compatible dependencies. Wake words, retrieval, visible screen capture, and bounded multi-step planning remain separate future features.

WINDOWS OFFICIAL REFERENCES

[WPF and desktop UI](#)

[Self-contained .NET deployment](#)

[Vosk models and licenses](#)

[Windows DPAPI](#)

[GitHub Releases](#)